



DYNAMIC PROGRAMMING

MARZIEH ESKANDARI
NJIT

DYNAMIC PROGRAMMING

- Where did the (strange) name Dynamic Programming come from?

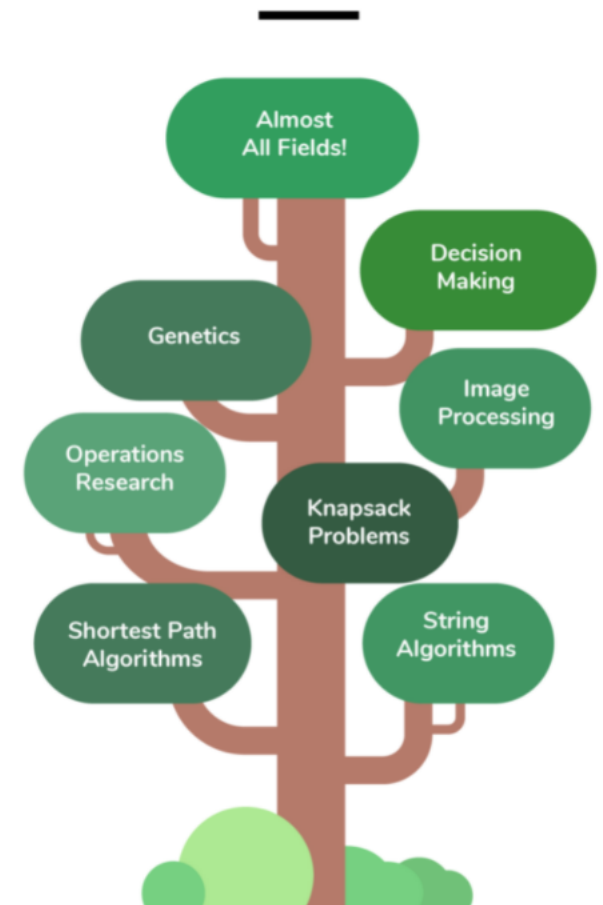
The inventor of DP-Richard **Bellman**- says: “I spent the Fall quarter (of **1950**) at RAND. My first task was to find a **name for multistage decision processes**. An interesting question is, ‘Where did the name, dynamic programming, come from?’ The 1950s were not good years for mathematical research. We had a very interesting gentleman in Washington named **Wilson**. He was Secretary of Defense, and he actually had a pathological fear and hatred of the word, research. I’m not using the term lightly; I’m using it precisely. His face would suffuse, he would turn red, and he would get violent if people used the term, research, in his presence. You can imagine how he felt, then, about the term, mathematical. The RAND Corporation was employed by the **Air Force**, and the Air Force had Wilson as its boss, essentially. Hence, I felt I had to do something to shield Wilson and the Air Force from the fact that I was really doing mathematics inside the RAND Corporation. What title, what name, could I choose? In the first place I was interested in planning, in decision making, in thinking. But planning, is not a good word for various reasons. I decided therefore to use the word, ‘programming.’ I wanted to get across the idea that this was dynamic, this was multistage, this was time-varying—I thought, let’s kill two birds with one stone. Let’s take a word that has an absolutely precise meaning, namely dynamic, in the classical physical sense. It also has a very interesting property as an adjective, and that is it’s impossible to use the word, dynamic, in a pejorative sense. Try thinking of some combination that will possibly give it a pejorative meaning. It’s impossible. Thus, I thought dynamic programming was a good name. It was something not even a Congressman could object to. So I used it as an umbrella for my activities”.



DEFINITION

- **Dynamic Programming (DP)** is a problem-solving approach helps find the best or most efficient solution to a problem by **systematically exploring all possible solutions**. It involves breaking a problem into smaller subproblems, solving each once, and storing their results to avoid redundant work.
- This approach improves efficiency, often reducing exponential time complexity to polynomial.

Applications of Dynamic Programming



DYNAMIC PROGRAMING:APPROACHES

❑ **Memoization:** This involves solving the problem **recursively** and storing the results of subproblems in a cache (storage) to avoid redundant computations.

1. Start with the original problem and break it down recursively.
2. Use a cache (e.g., a dictionary or array) to store already computed subproblem results.
3. Reuse results from the cache if the same subproblem is encountered again.

❑ **Tabulation:** This involves solving the problem **iteratively** by building a table (usually an array) from the smallest subproblems to the larger ones.

1. Identify the base cases and initialize a table to store solutions.
2. Iteratively fill in the table, solving subproblems in a specific order.
3. Use previously computed values to build up to the solution of the original problem.

DYNAMIC PROGRAMING:TABULATION

1. Define the “restricted” small **subproblems**.
2. Define a “Table” to store the solutions to the subproblems.
3. Find a **relation** between the solution of a larger problem and the solutions of smaller problems.
4. Use an **inductive method** to construct the solution to the main problem: Starting from the base cases (smallest subproblems), iteratively fill the table using this **relation**.

DP VS. DC

□ Similarity:

1. Both break a problem into subproblems:
They solve the main problem by solving smaller, related subproblems.
2. Both use recursion (either directly or indirectly).
3. Both aim to reduce the problem's time complexity by reusing solutions or simplifying the problem structure.

□ Difference:

1. In DC, subproblems are mostly **independent** whereas in DP subproblems are **overlapping**
2. DC does not reuse solutions; recalculates them as needed whereas DP stores and reuses solutions to subproblems (memoization/tabulation).
3. DC may have exponential time if not optimized while DP is typically more efficient (polynomial time) due to memoization.

EXAMPLE 1: FIBONACCI'S SERIES

- $F(1) = F(2) = 1, \quad F(n) = F(n-1) + F(n-2)$

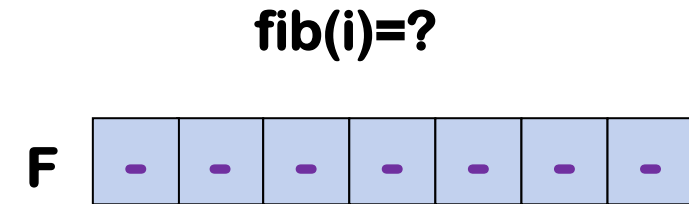
```
fib(n):  
    if n==1 or n==2 then  
        return 1  
    return fib(n-1)+fib(n-2)
```

$O(2^n)$

FIBONACCI'S SERIES: DP APPROACH

1. Subproblems: $fib(i): 0 \leq i \leq n$
2. Define a “Table” to store $fib(i)$
3. Find a **relation** : $fib(i) = fib(i - 1) + fib(i - 2)$
4. Inductive method:

```
F[0]=F[1]=1
for i=2 to n do:
    F[i]=F[i-1]+F[i-2]
```



FIBONACCI'S SERIES: DP-TABULATION

```
fib(n):  
    F[0]=F[1]=1  
  
    for i=2 to n do:  
        F[i]=F[i-1]+F[i-2]
```

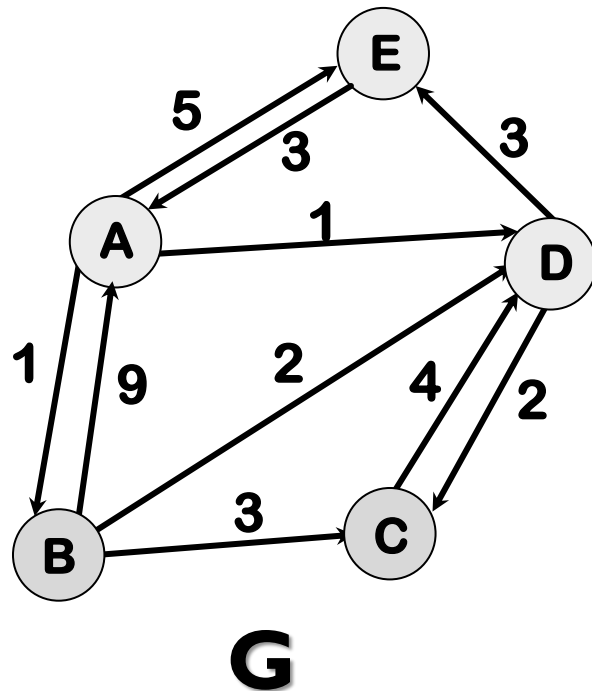
$O(n)$

fib(5)=?

F	-	-	-	-	-	-
F	1	1	-	-	-	-
F	1	1	2	-	-	-
F	1	1	2	3	-	-
F	1	1	2	3	5	-

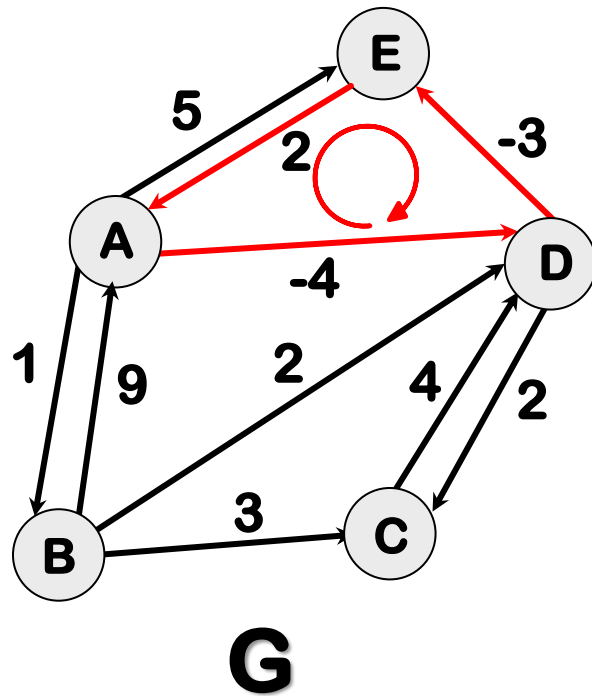
EXAMPLE 3: FLOYD'S METHOD

- Given a weighted directed graph with positive or negative edge weights (but with no negative cycles), find the shortest path between every pair of nodes.



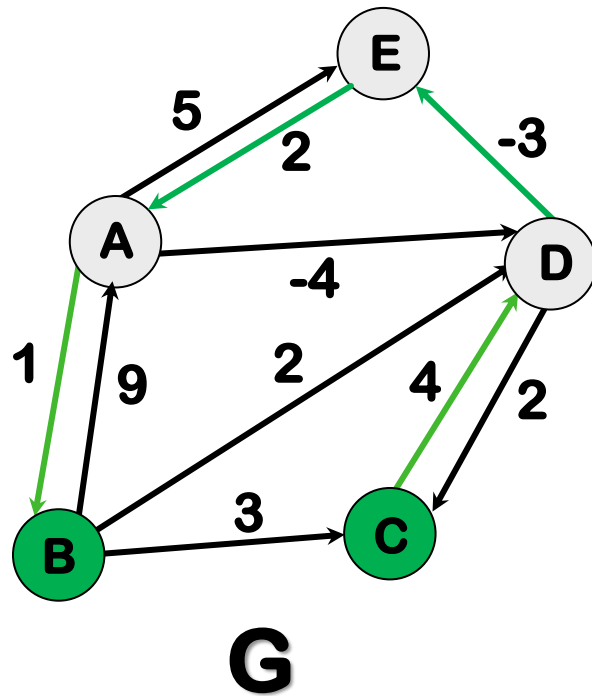
EXAMPLE 3: FLOYD'S METHOD

- Given a weighted directed graph with positive or negative edge weights (but with no negative cycles), find the shortest path between every pair of nodes.



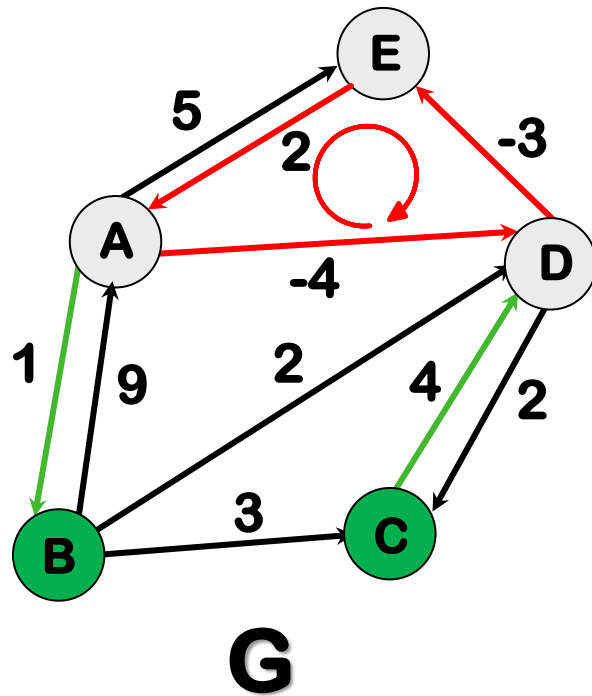
EXAMPLE 3: FLOYD'S METHOD

- Given a weighted directed graph with positive or negative edge weights (but with no negative cycles), find the shortest path between every pair of nodes.



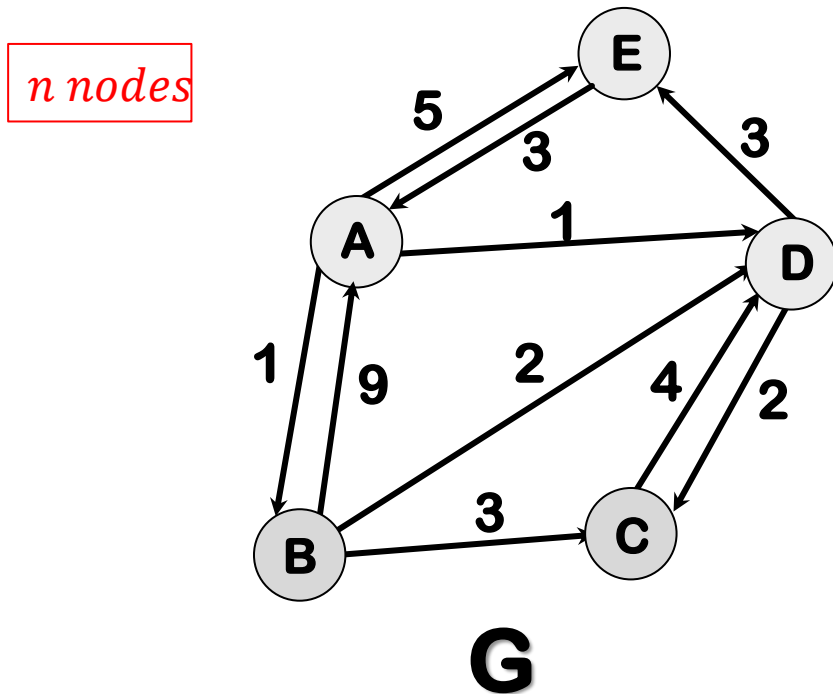
EXAMPLE 3: FLOYD'S METHOD

- Given a weighted directed graph with positive or negative edge weights (but with no negative cycles), find the shortest path between every pair of nodes.



EXAMPLE 3: FLOYD'S METHOD

- Given a weighted directed graph with positive or negative edge weights (but with no negative cycles), find the shortest path between every pair of nodes.

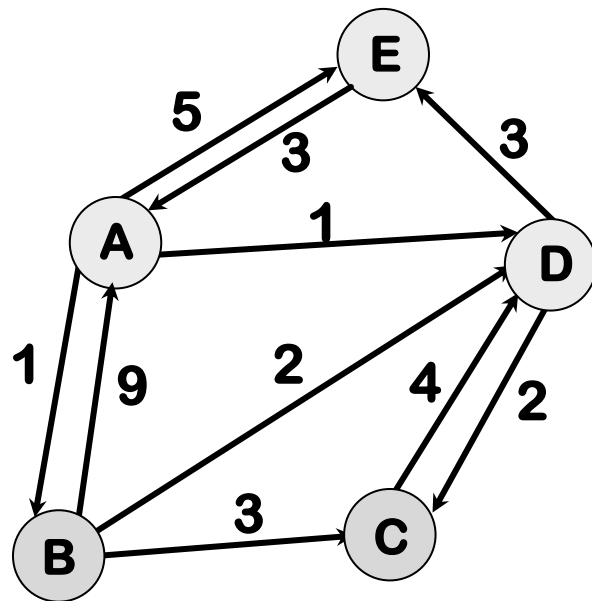


A to B
A to C
A to D
A to E
B to A
B to C
B to D
B to E
C to A
C to B

C to D
C to E
D to A
D to B
D to C
D to E
E to A
E to B
E to C
E to D

EXAMPLE 3: FLOYD'S METHOD

- Given a weighted directed graph, find the shortest path between every pair of nodes.



G

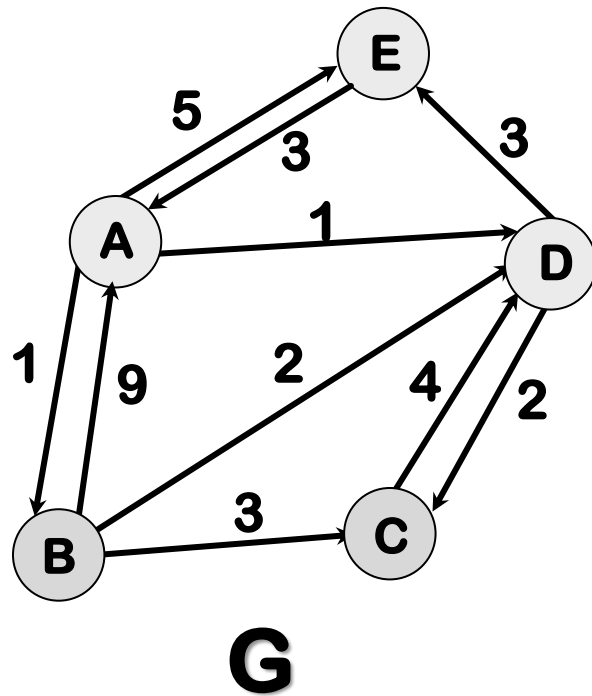


A-B
A-D-C
A-D
A-D-E
B-D-E-A
B-C
B-D
B-D-E
C-D-E-A
C-D-E-A-B

C-D
C-D-E
D-E-A
D-E-A-B
D-C
D-E
E-A
E-A-B
E-A-D-C
E-A-D

EXAMPLE 3: FLOYD'S METHOD

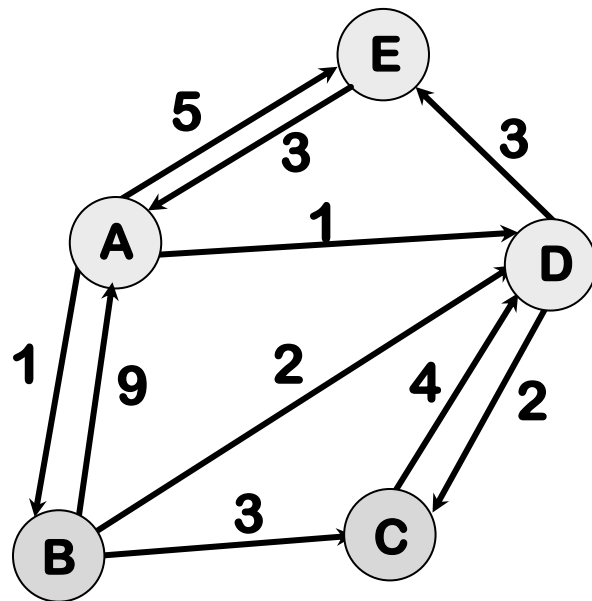
- Given a weighted directed graph, find the shortest path between every pair of nodes.



Lengths of all 20 shortest paths

EXAMPLE 3: FLOYD'S METHOD

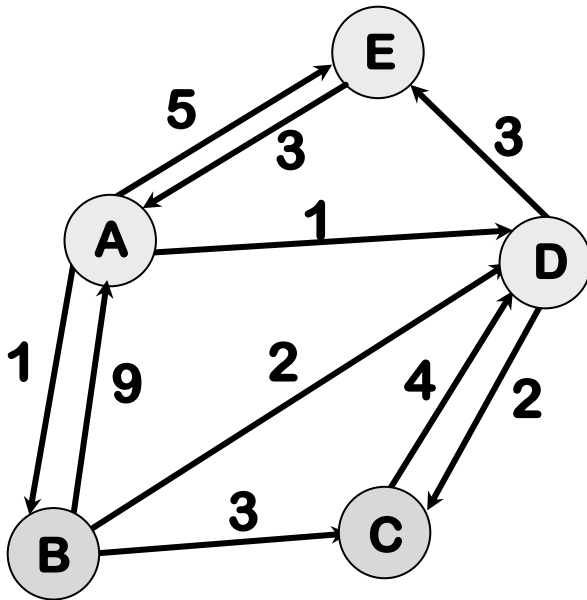
- Given a weighted directed graph, find the **lengths of the shortest paths** between all pairs of nodes.



A to B: 1
A to C: 3
A to D: 1
A to E: 4
B to A: 8
B to C: 3
B to D: 2
B to E: 5
C to A: 10
C to B: 11

C to D: 4
C to E: 7
D to A: 6
D to B: 7
D to C: 2
D to E: 3
E to A: 3
E to B: 4
E to C: 6
E to D: 4

GRAPH: REPRESENTATION



G

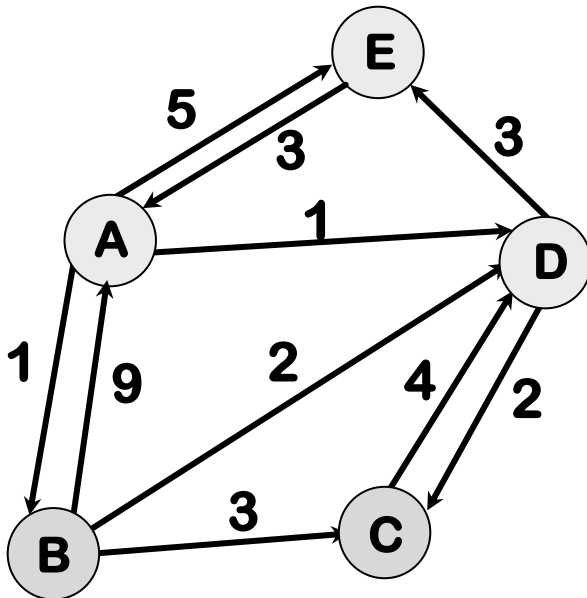


	A	B	C	D	E
A	0	1	∞	1	5
B	9	0	3	2	∞
C	∞	∞	0	4	∞
D	∞	∞	2	0	3
E	3	∞	∞	∞	0

W

$W[i][j]$ = weight of edge from vertex i to j

SHORTEST PATHS: REPRESENTATION



G

$O(n^3)$

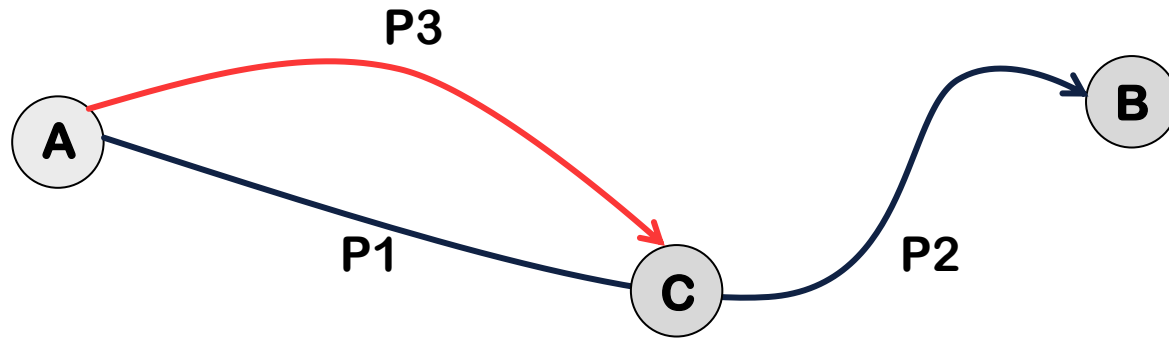


	A	B	C	D	E
A	0	1	3	1	4
B	8	0	3	2	5
C	10	11	0	4	7
D	6	7	2	0	3
E	3	4	6	4	0

D

$D[i][j]$ = length of shortest path from vertex i to j

OPTIMAL SUBSTRUCTURE



$$SP(A, B) = P1 + P2$$

If P1 is not the shortest path between A and B,
let P3 is the shortest path between A and C.

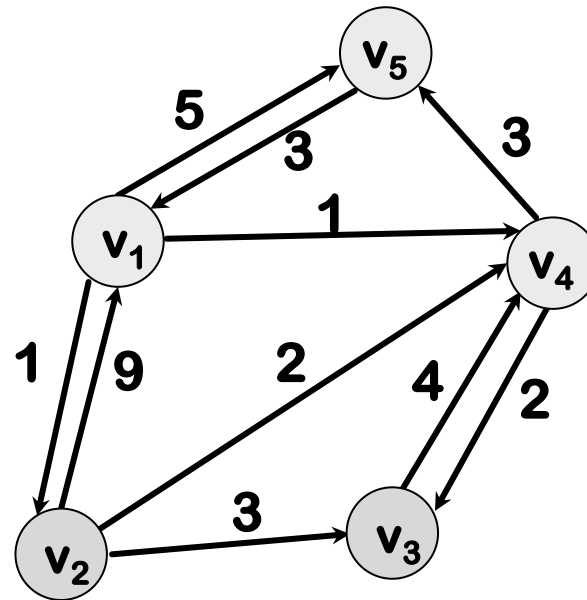
So we have: $|P3| + |P2| < |P1| + |P2| = |SP(A, B)|$. That is a contradiction.

ORIGINAL PROBLEM

- Find the shortest path between all pairs of vertices (v_i, v_j) in the given graph.
- Subproblem=original problem + a restriction!

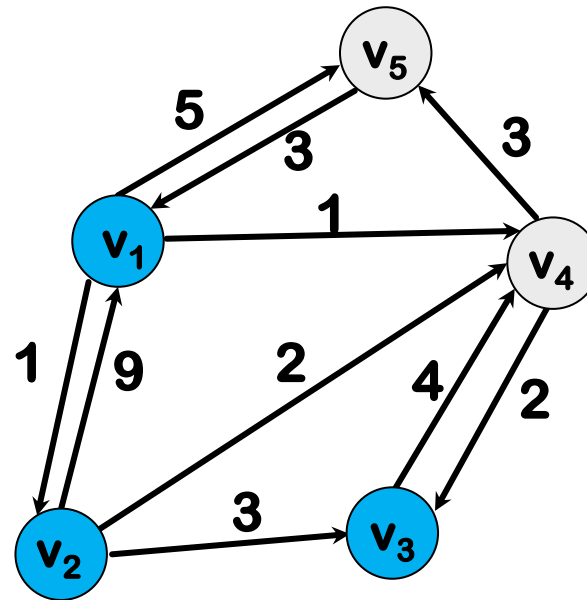
DEFINING SUBPROBLEMS

- Find the shortest path between all pairs of vertices (v_i, v_j) considering only intermediate vertices from the set $\{v_1, v_2, \dots, v_k\}$.



DEFINING SUBPROBLEMS

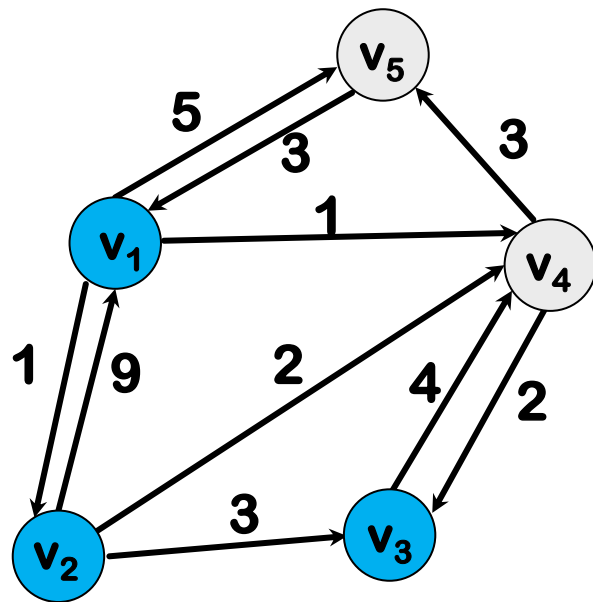
- Find the shortest path between all pairs of vertices (v_i, v_j) considering only intermediate vertices from the set $\{v_1, v_2, v_3\}$.



k=3

SUBPROBLEMS: EXAMPLE

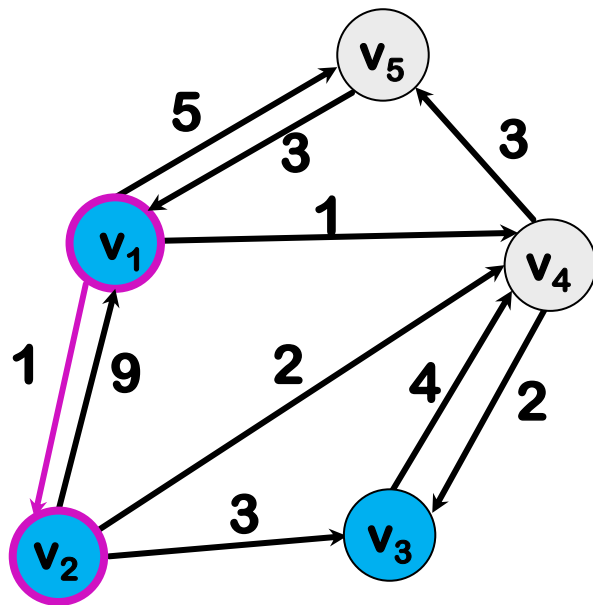
$d(v_i, v_j)$: length of shortest path from v_i to v_j



SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j

$$d(v_1, v_2) = 1$$

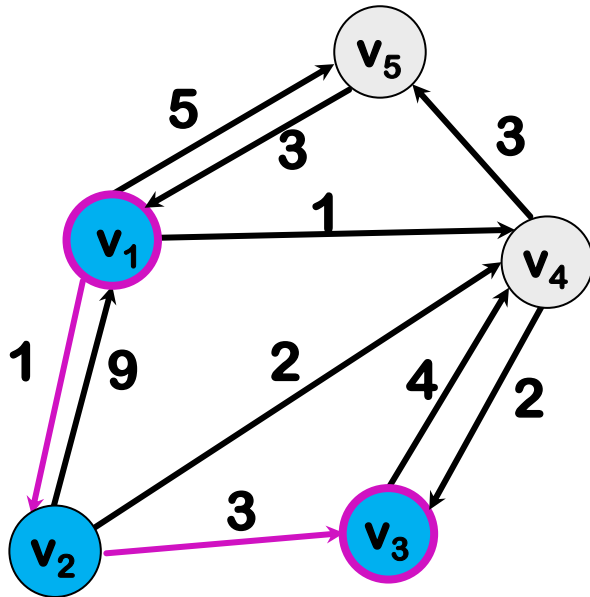


SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j

$$d(v_1, v_2) = 1$$

$$d(v_1, v_3) = 4$$



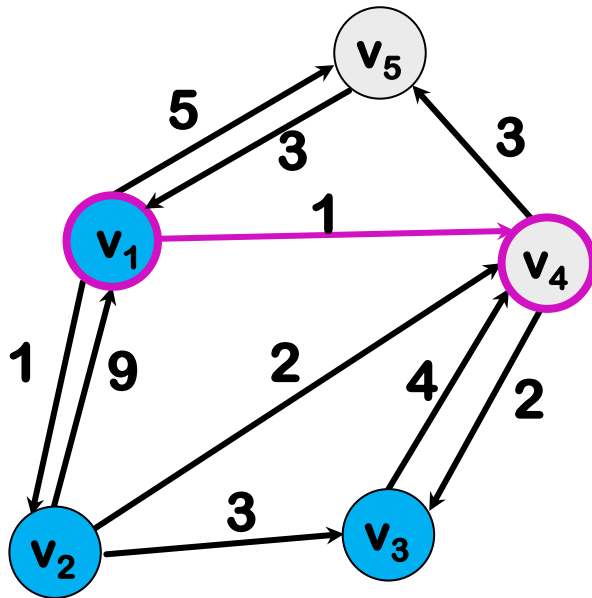
SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j

$$d(v_1, v_2) = 1$$

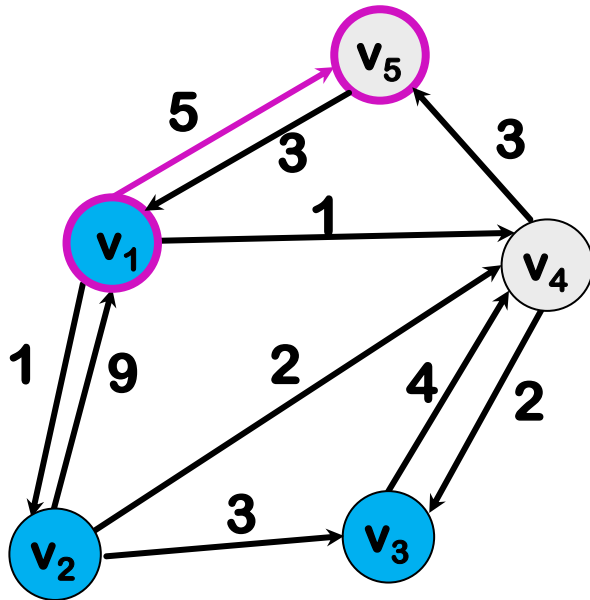
$$d(v_1, v_3) = 4$$

$$d(v_1, v_4) = 1$$



SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j



$$d(v_1, v_2) = 1$$

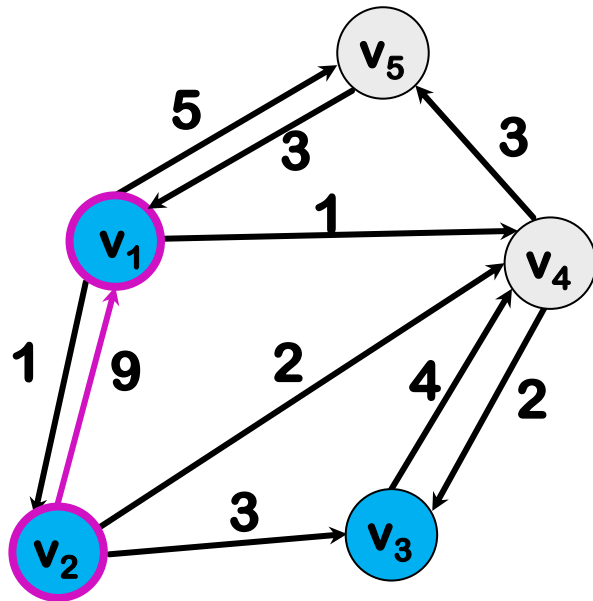
$$d(v_1, v_3) = 4$$

$$d(v_1, v_4) = 1$$

$$d(v_1, v_5) = 5$$

SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j



$$d(v_1, v_2) = 1$$

$$d(v_1, v_3) = 4$$

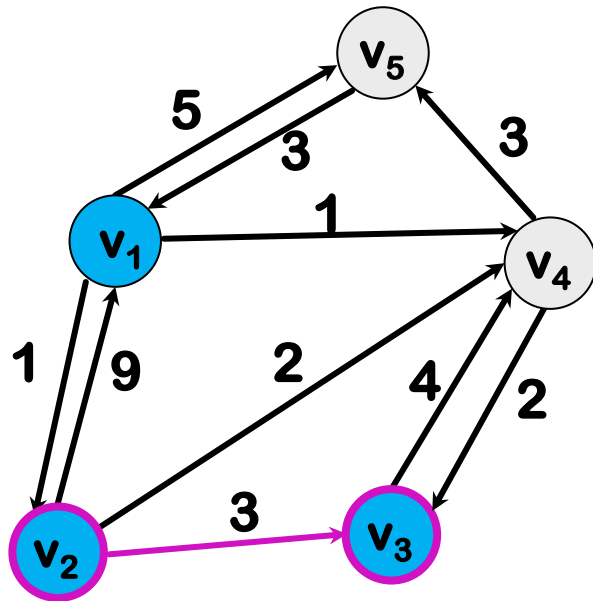
$$d(v_1, v_4) = 1$$

$$d(v_1, v_5) = 5$$

$$d(v_2, v_1) = 9$$

SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j



$$d(v_1, v_2) = 1$$

$$d(v_1, v_3) = 4$$

$$d(v_1, v_4) = 1$$

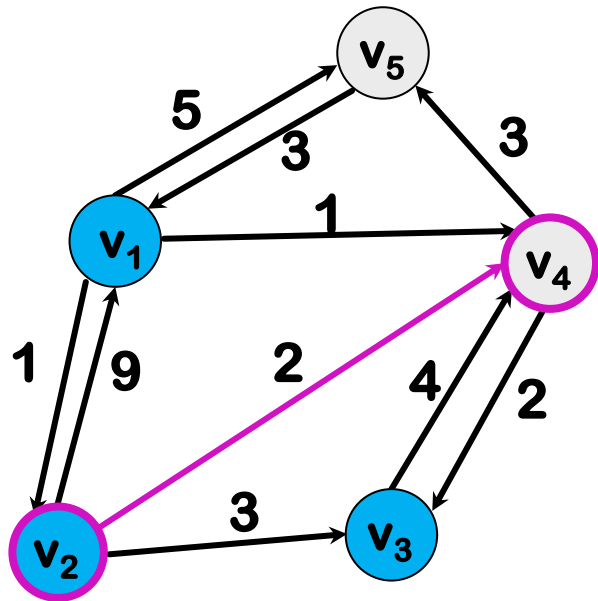
$$d(v_1, v_5) = 5$$

$$d(v_2, v_1) = 9$$

$$d(v_2, v_3) = 3$$

SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j



$$d(v_1, v_2) = 1$$

$$d(v_1, v_3) = 4$$

$$d(v_1, v_4) = 1$$

$$d(v_1, v_5) = 5$$

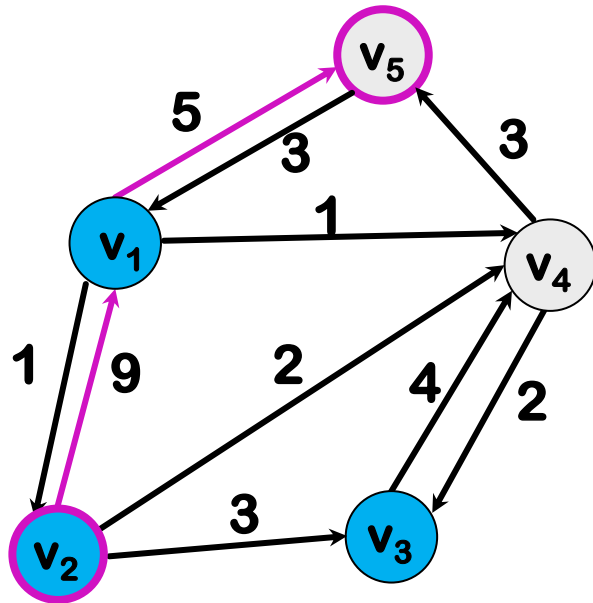
$$d(v_2, v_1) = 9$$

$$d(v_2, v_3) = 3$$

$$d(v_2, v_4) = 2$$

SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j



$$d(v_1, v_2) = 1$$

$$d(v_1, v_3) = 4$$

$$d(v_1, v_4) = 1$$

$$d(v_1, v_5) = 5$$

$$d(v_2, v_1) = 9$$

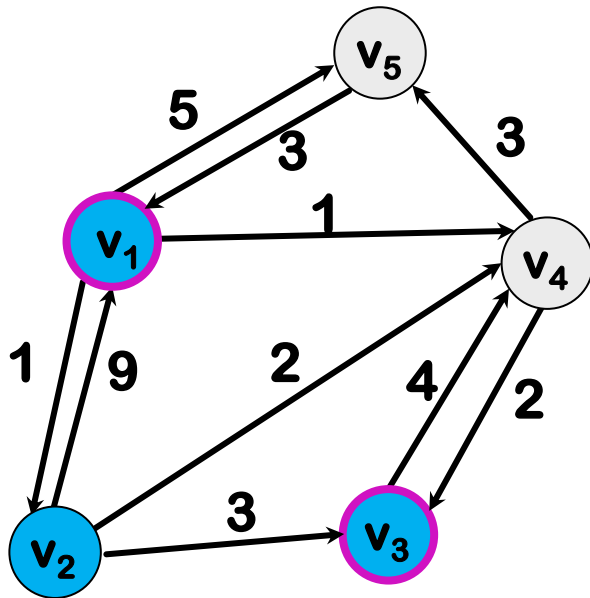
$$d(v_2, v_3) = 3$$

$$d(v_2, v_4) = 2$$

$$d(v_2, v_5) = 14$$

SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j



$$d(v_1, v_2) = 1$$

$$d(v_1, v_3) = 4$$

$$d(v_1, v_4) = 1$$

$$d(v_1, v_5) = 5$$

$$d(v_2, v_1) = 9$$

$$d(v_2, v_3) = 3$$

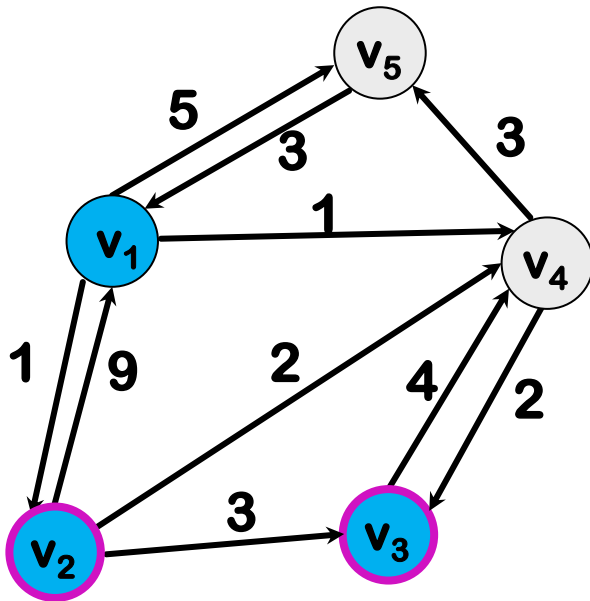
$$d(v_2, v_4) = 2$$

$$d(v_2, v_5) = 14$$

$$d(v_3, v_1) = \infty$$

SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j



$$d(v_1, v_2) = 1$$

$$d(v_1, v_3) = 4$$

$$d(v_1, v_4) = 1$$

$$d(v_1, v_5) = 5$$

$$d(v_2, v_1) = 9$$

$$d(v_2, v_3) = 3$$

$$d(v_2, v_4) = 2$$

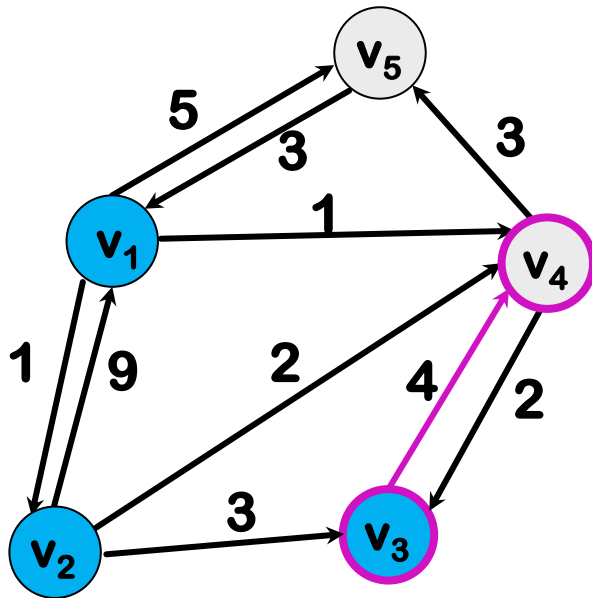
$$d(v_2, v_5) = 14$$

$$d(v_3, v_1) = \infty$$

$$d(v_3, v_2) = \infty$$

SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j



$$d(v_1, v_2) = 1$$

$$d(v_3, v_4) = 4$$

$$d(v_1, v_3) = 4$$

$$d(v_1, v_4) = 1$$

$$d(v_1, v_5) = 5$$

$$d(v_2, v_1) = 9$$

$$d(v_2, v_3) = 3$$

$$d(v_2, v_4) = 2$$

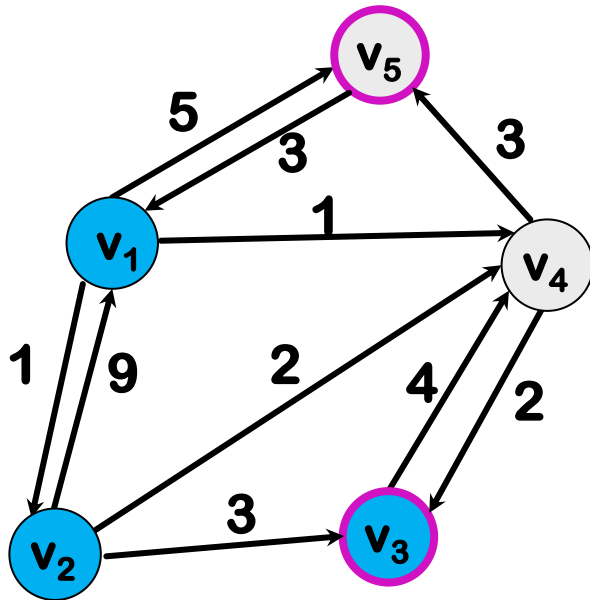
$$d(v_2, v_5) = 14$$

$$d(v_3, v_1) = \infty$$

$$d(v_3, v_2) = \infty$$

SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j



$$d(v_1, v_2) = 1$$

$$d(v_1, v_3) = 4$$

$$d(v_1, v_4) = 1$$

$$d(v_1, v_5) = 5$$

$$d(v_2, v_1) = 9$$

$$d(v_2, v_3) = 3$$

$$d(v_2, v_4) = 2$$

$$d(v_2, v_5) = 14$$

$$d(v_3, v_1) = \infty$$

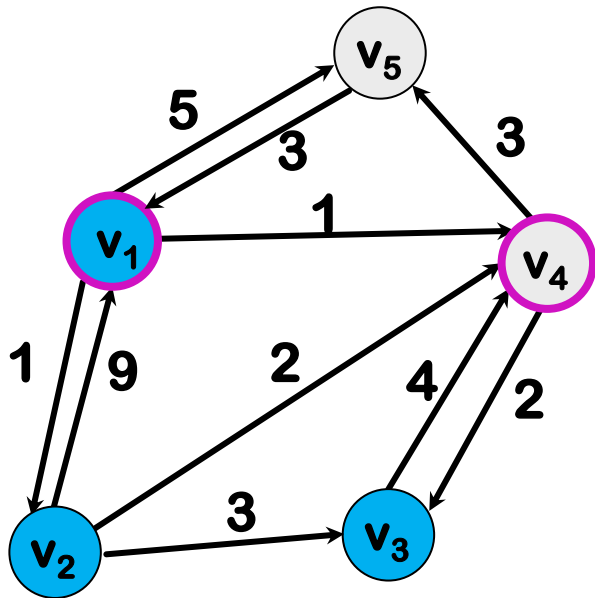
$$d(v_3, v_2) = \infty$$

$$d(v_3, v_4) = 4$$

$$d(v_3, v_5) = \infty$$

SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j



$$d(v_1, v_2) = 1$$

$$d(v_1, v_3) = 4$$

$$d(v_1, v_4) = 1$$

$$d(v_1, v_5) = 5$$

$$d(v_2, v_1) = 9$$

$$d(v_2, v_3) = 3$$

$$d(v_2, v_4) = 2$$

$$d(v_2, v_5) = 14$$

$$d(v_3, v_1) = \infty$$

$$d(v_3, v_2) = \infty$$

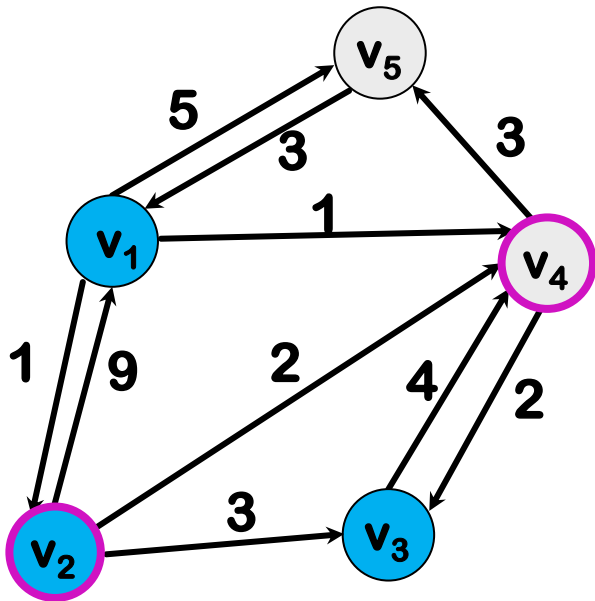
$$d(v_3, v_4) = 4$$

$$d(v_3, v_5) = \infty$$

$$d(v_4, v_1) = \infty$$

SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j



$$d(v_1, v_2) = 1$$

$$d(v_1, v_3) = 4$$

$$d(v_1, v_4) = 1$$

$$d(v_1, v_5) = 5$$

$$d(v_2, v_1) = 9$$

$$d(v_2, v_3) = 3$$

$$d(v_2, v_4) = 2$$

$$d(v_2, v_5) = 14$$

$$d(v_3, v_1) = \infty$$

$$d(v_3, v_2) = \infty$$

$$d(v_3, v_4) = 4$$

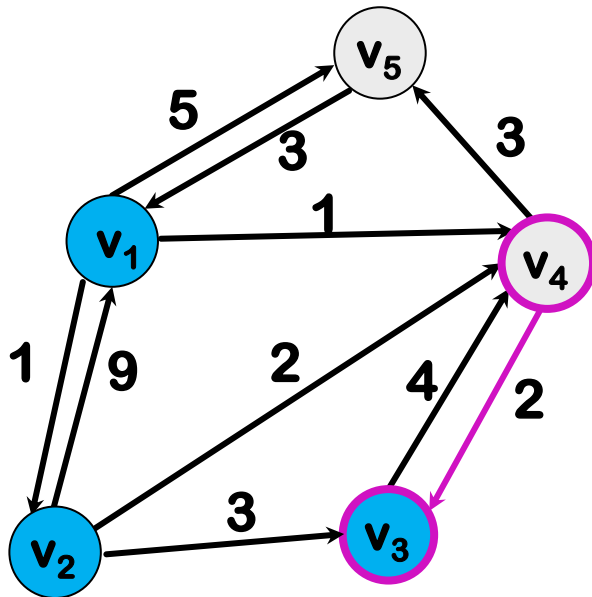
$$d(v_3, v_5) = \infty$$

$$d(v_4, v_1) = \infty$$

$$d(v_4, v_2) = \infty$$

SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j



$$d(v_1, v_2) = 1$$

$$d(v_1, v_3) = 4$$

$$d(v_1, v_4) = 1$$

$$d(v_1, v_5) = 5$$

$$d(v_2, v_1) = 9$$

$$d(v_2, v_3) = 3$$

$$d(v_2, v_4) = 2$$

$$d(v_2, v_5) = 14$$

$$d(v_3, v_1) = \infty$$

$$d(v_3, v_2) = \infty$$

$$d(v_3, v_4) = 4$$

$$d(v_3, v_5) = \infty$$

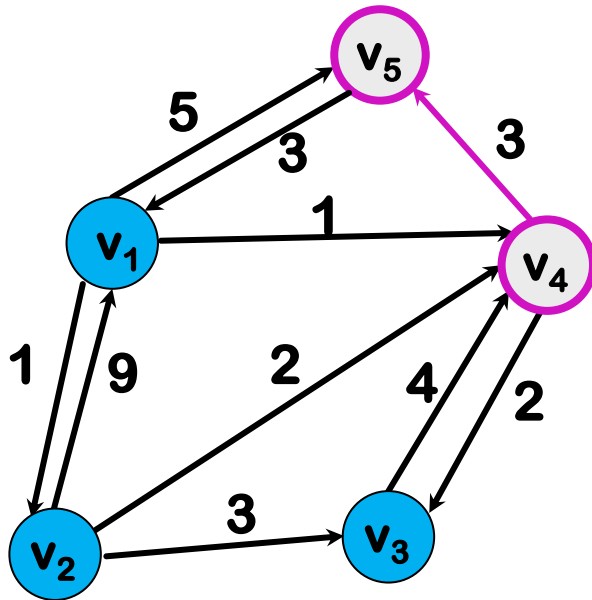
$$d(v_4, v_1) = \infty$$

$$d(v_4, v_2) = \infty$$

$$d(v_4, v_3) = 2$$

SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j



$$d(v_1, v_2) = 1$$

$$d(v_1, v_3) = 4$$

$$d(v_1, v_4) = 1$$

$$d(v_1, v_5) = 5$$

$$d(v_2, v_1) = 9$$

$$d(v_2, v_3) = 3$$

$$d(v_2, v_4) = 2$$

$$d(v_2, v_5) = 14$$

$$d(v_3, v_1) = \infty$$

$$d(v_3, v_2) = \infty$$

$$d(v_3, v_4) = 4$$

$$d(v_3, v_5) = \infty$$

$$d(v_4, v_1) = \infty$$

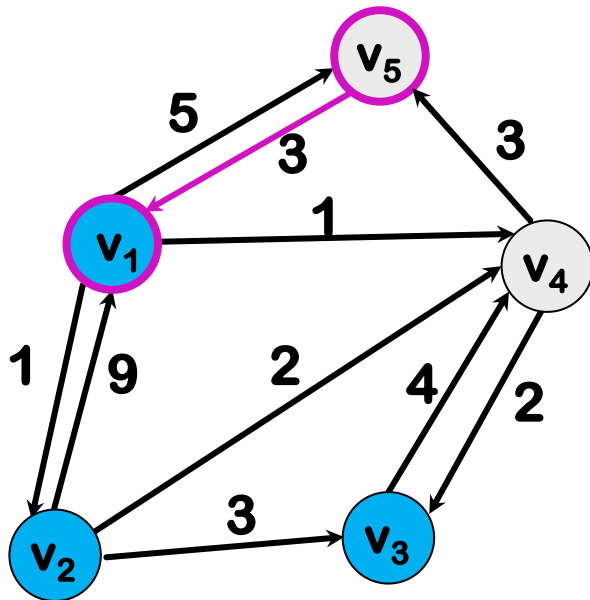
$$d(v_4, v_2) = \infty$$

$$d(v_4, v_3) = 2$$

$$d(v_4, v_5) = 3$$

SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j



$$d(v_1, v_2) = 1$$

$$d(v_1, v_3) = 4$$

$$d(v_1, v_4) = 1$$

$$d(v_1, v_5) = 5$$

$$d(v_2, v_1) = 9$$

$$d(v_2, v_3) = 3$$

$$d(v_2, v_4) = 2$$

$$d(v_2, v_5) = 14$$

$$d(v_3, v_1) = \infty$$

$$d(v_3, v_2) = \infty$$

$$d(v_3, v_4) = 4$$

$$d(v_3, v_5) = \infty$$

$$d(v_4, v_1) = \infty$$

$$d(v_4, v_2) = \infty$$

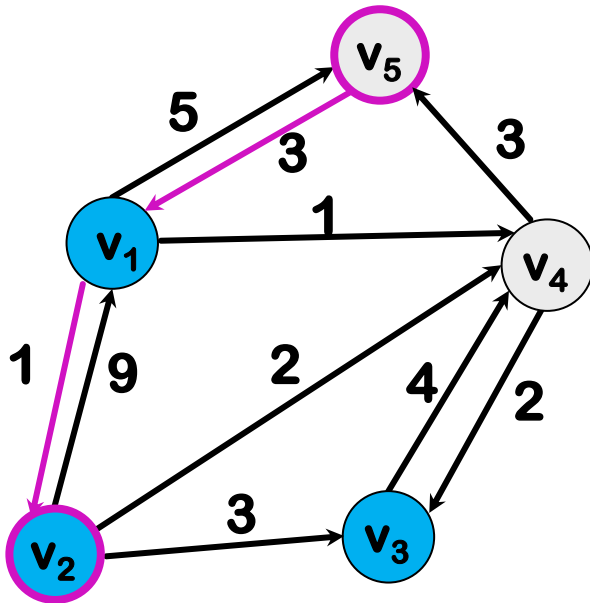
$$d(v_4, v_3) = 2$$

$$d(v_4, v_5) = 3$$

$$d(v_5, v_1) = 3$$

SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j



$$d(v_1, v_2) = 1$$

$$d(v_1, v_3) = 4$$

$$d(v_1, v_4) = 1$$

$$d(v_1, v_5) = 5$$

$$d(v_2, v_1) = 9$$

$$d(v_2, v_3) = 3$$

$$d(v_2, v_4) = 2$$

$$d(v_2, v_5) = 14$$

$$d(v_3, v_1) = \infty$$

$$d(v_3, v_2) = \infty$$

$$d(v_3, v_4) = 4$$

$$d(v_3, v_5) = \infty$$

$$d(v_4, v_1) = \infty$$

$$d(v_4, v_2) = \infty$$

$$d(v_4, v_3) = 2$$

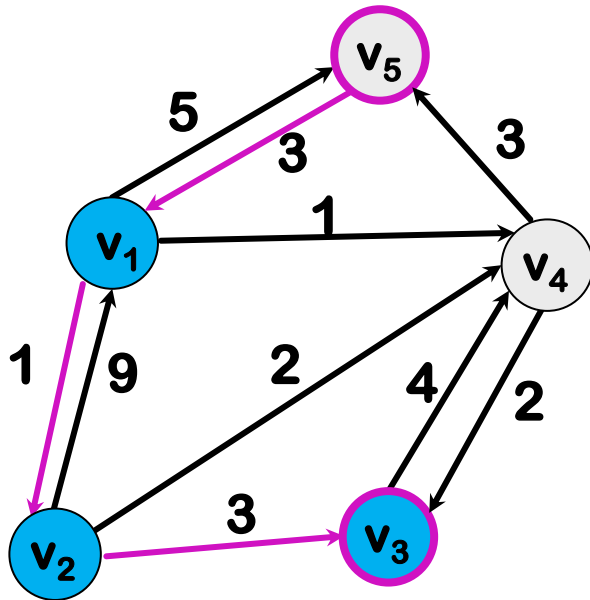
$$d(v_4, v_5) = 3$$

$$d(v_5, v_1) = 3$$

$$d(v_5, v_2) = 4$$

SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j



$$d(v_1, v_2) = 1$$

$$d(v_1, v_3) = 4$$

$$d(v_1, v_4) = 1$$

$$d(v_1, v_5) = 5$$

$$d(v_2, v_1) = 9$$

$$d(v_2, v_3) = 3$$

$$d(v_2, v_4) = 2$$

$$d(v_2, v_5) = 14$$

$$d(v_3, v_1) = \infty$$

$$d(v_3, v_2) = \infty$$

$$d(v_3, v_4) = 4$$

$$d(v_3, v_5) = \infty$$

$$d(v_4, v_1) = \infty$$

$$d(v_4, v_2) = \infty$$

$$d(v_4, v_3) = 2$$

$$d(v_4, v_5) = 3$$

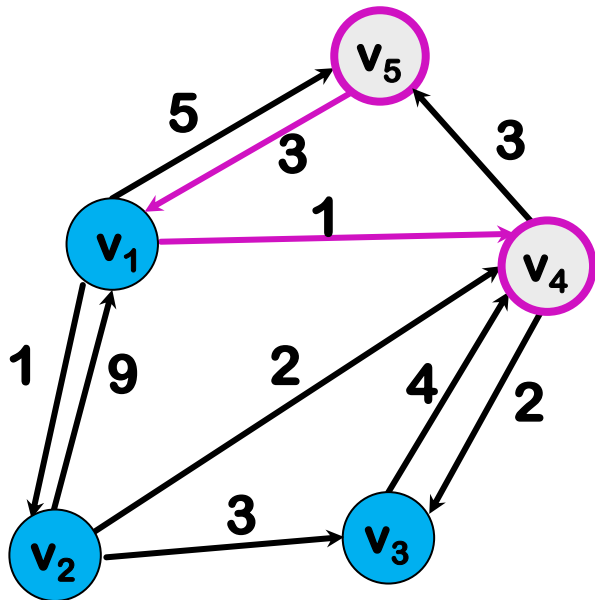
$$d(v_5, v_1) = 2$$

$$d(v_5, v_2) = 4$$

$$d(v_5, v_3) = 7$$

SUBPROBLEMS: EXAMPLE

$d(v_i, v_j)$: length of shortest path from v_i to v_j



$$d(v_1, v_2) = 1$$

$$d(v_1, v_3) = 4$$

$$d(v_1, v_4) = 1$$

$$d(v_1, v_5) = 5$$

$$d(v_2, v_1) = 9$$

$$d(v_2, v_3) = 3$$

$$d(v_2, v_4) = 2$$

$$d(v_2, v_5) = 14$$

$$d(v_3, v_1) = \infty$$

$$d(v_3, v_2) = \infty$$

$$d(v_3, v_4) = 4$$

$$d(v_3, v_5) = \infty$$

$$d(v_4, v_1) = \infty$$

$$d(v_4, v_2) = \infty$$

$$d(v_4, v_3) = 2$$

$$d(v_4, v_5) = 3$$

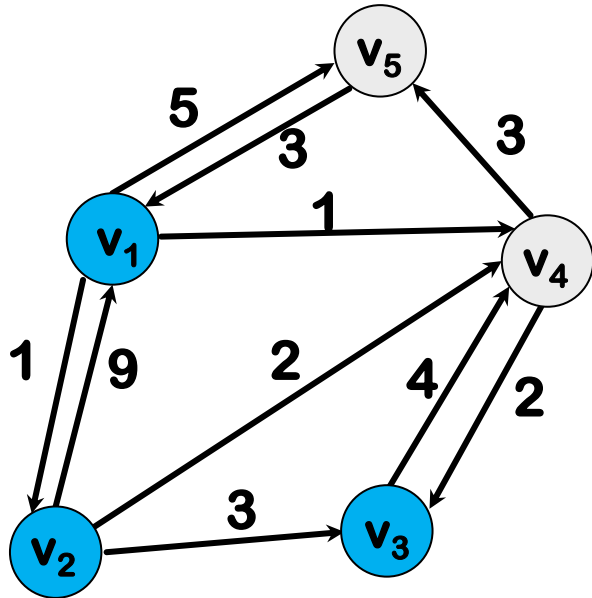
$$d(v_5, v_1) = 2$$

$$d(v_5, v_2) = 4$$

$$d(v_5, v_3) = 7$$

$$d(v_5, v_4) = 4$$

SUBPROBLEMS: REPRESENTATION



	v1	v2	v3	v4	v5
v1	0	1	4	1	5
v2	9	0	3	2	14
v3	∞	∞	0	4	∞
v4	∞	∞	2	0	3
v5	3	4	7	4	0

$\mathbf{D}^{(3)}$

$\mathbf{D}^{(k)}[i][j]$ = the length of the shortest path from vertex v_i to vertex v_j considering only intermediate vertices from the set $\{v_1, v_2, \dots, v_k\}$

NOTATIONS

$$\mathbf{D}^{(0)} =$$

NOTATIONS

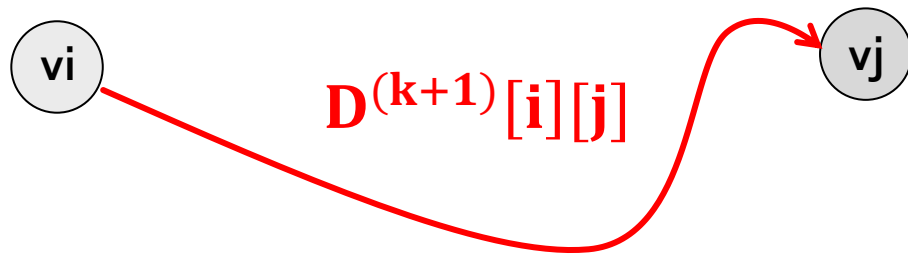
$$\mathbf{D}^{(0)} = \mathbf{W}$$

$$\mathbf{D}^{(n)} = \mathbf{D}$$

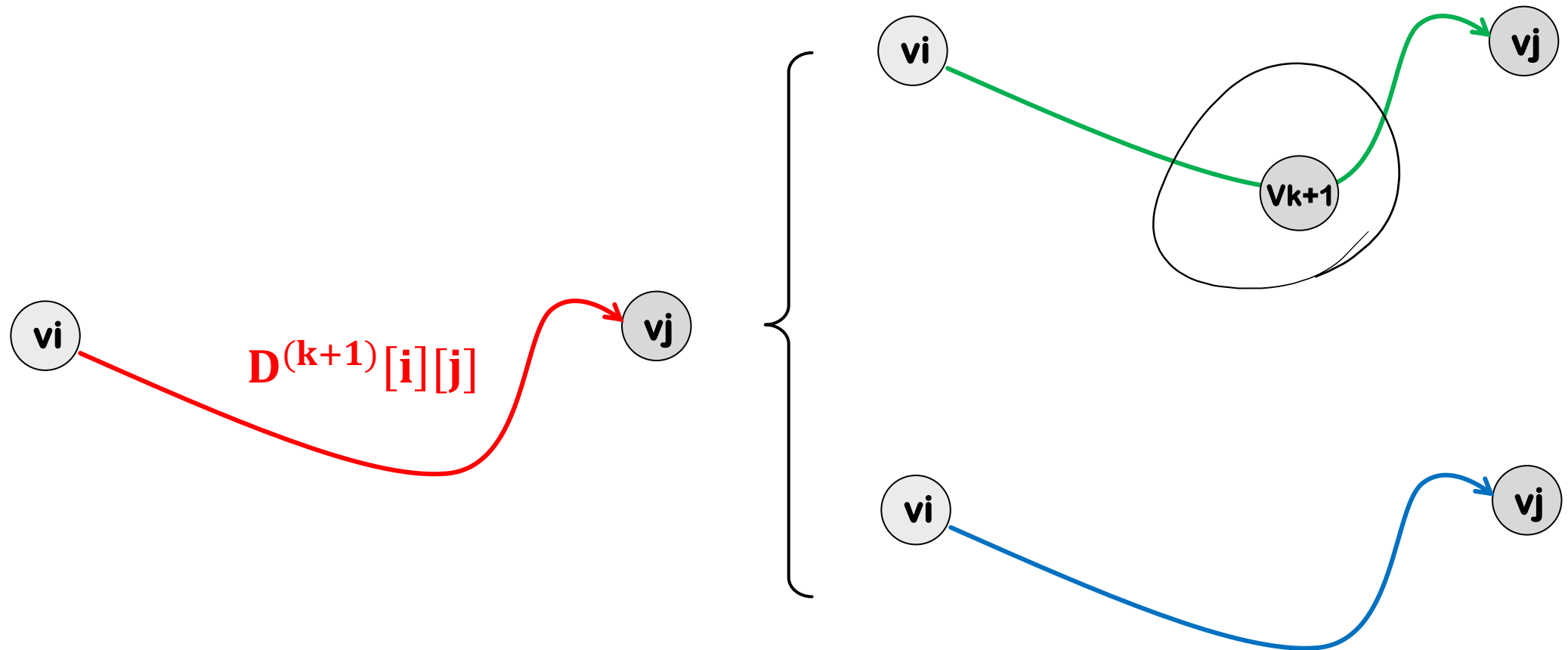
$$\mathbf{D}^{(k)} \longrightarrow \mathbf{D}^{(k+1)}$$

$$\mathbf{D}^{(0)} = \mathbf{W} \longrightarrow \mathbf{D}^{(1)} \longrightarrow \mathbf{D}^{(2)} \longrightarrow \mathbf{D}^{(3)} \longrightarrow \dots \longrightarrow \mathbf{D}^{(n)} = \mathbf{D}$$

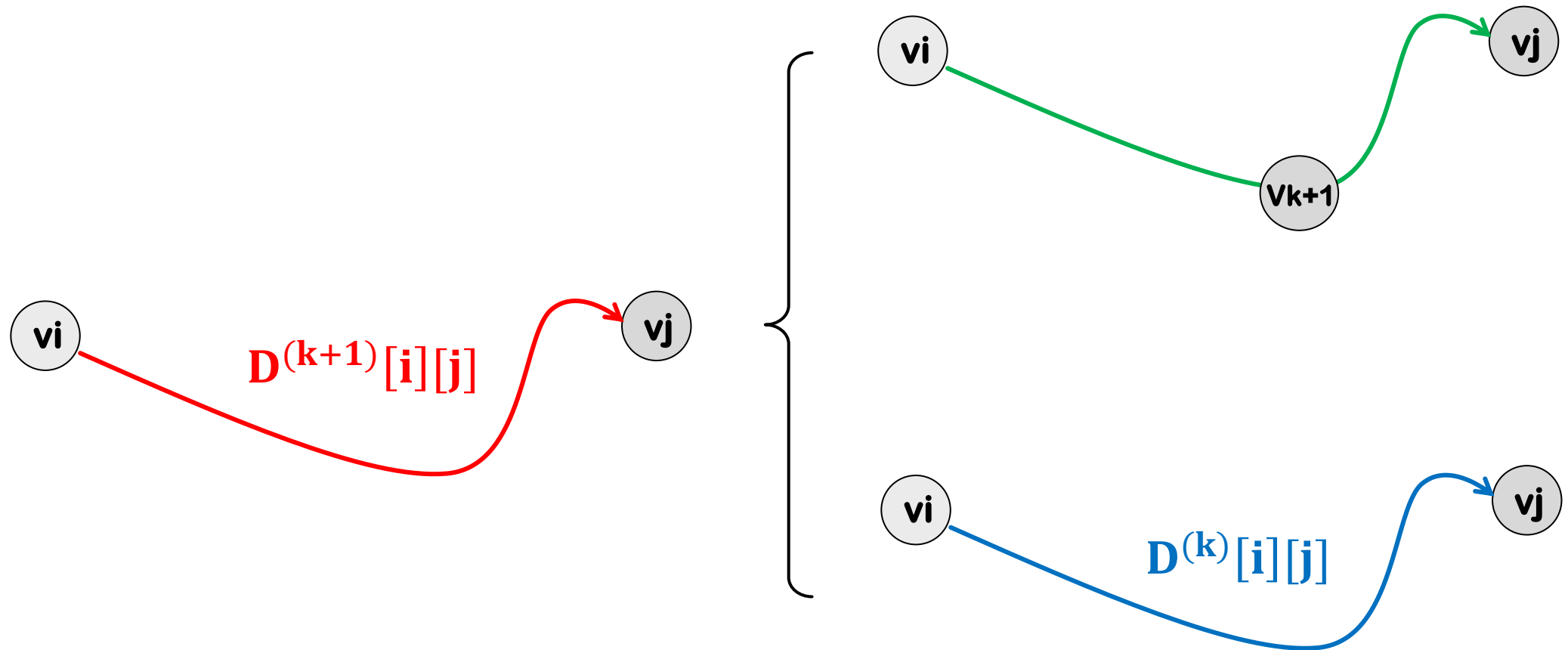
CALCULATING DS



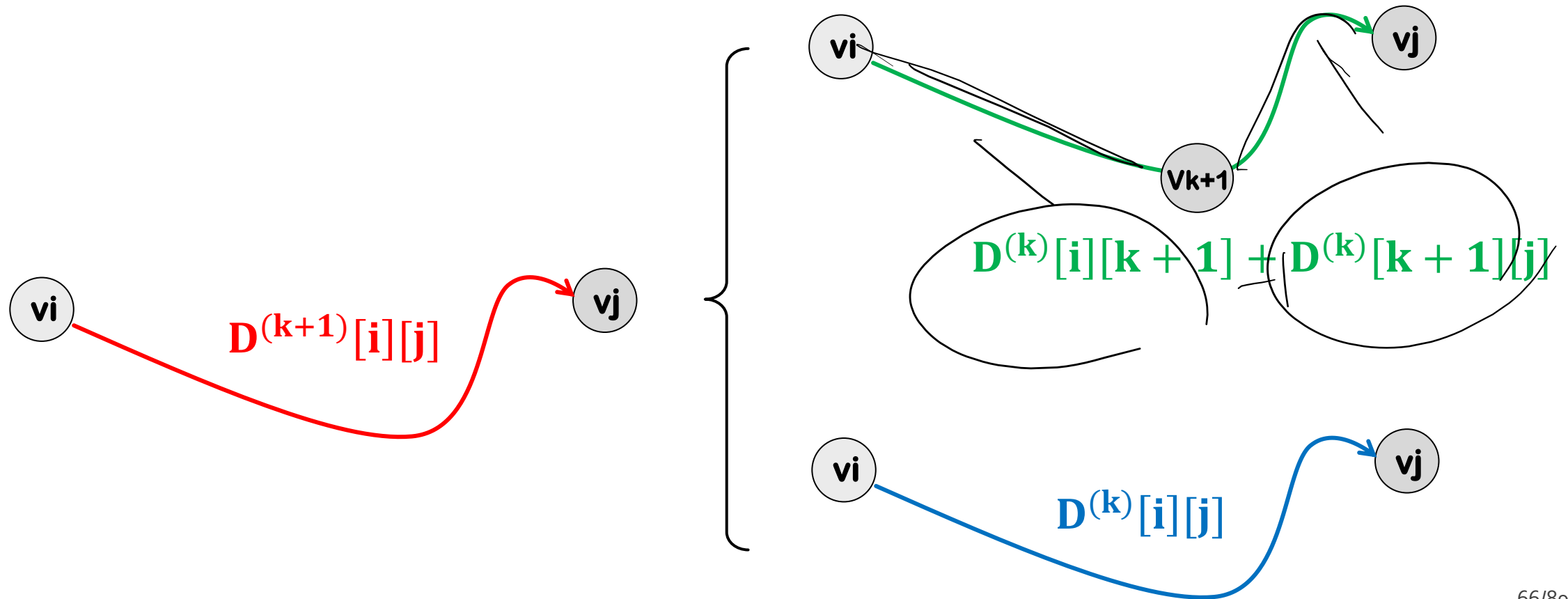
CALCULATING DS



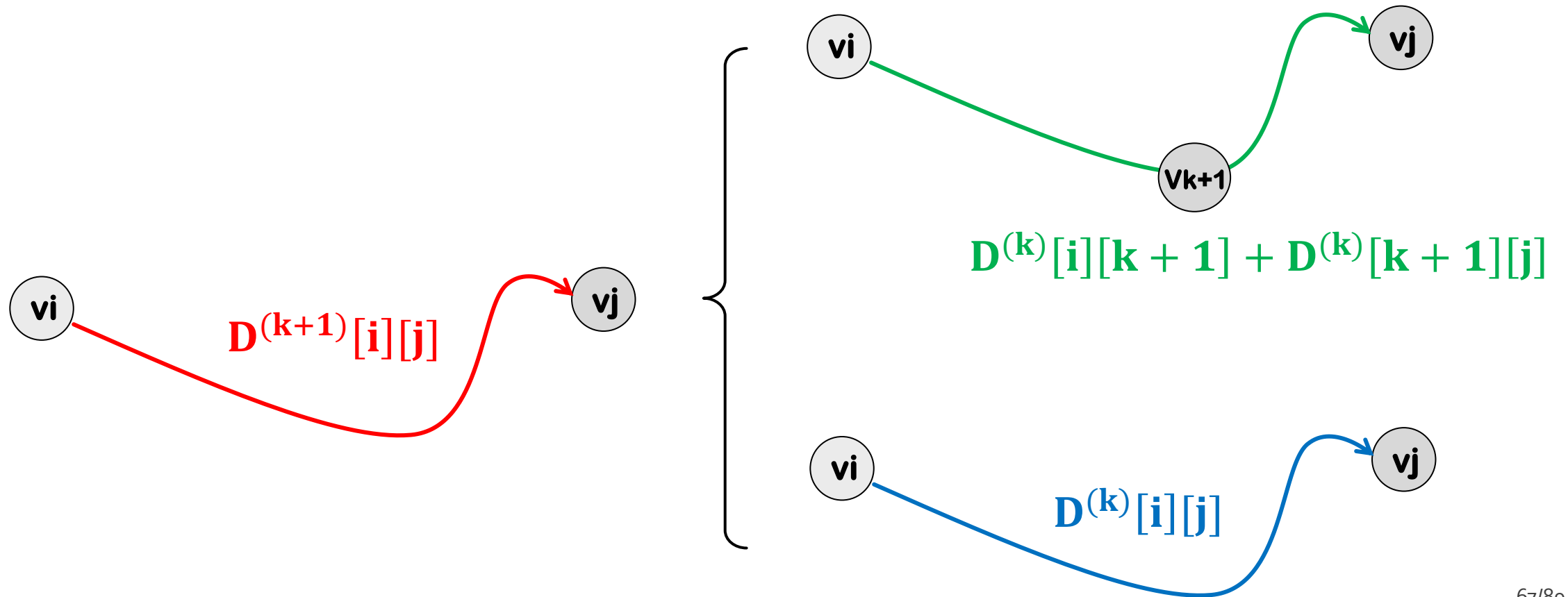
CALCULATING DS



CALCULATING DS

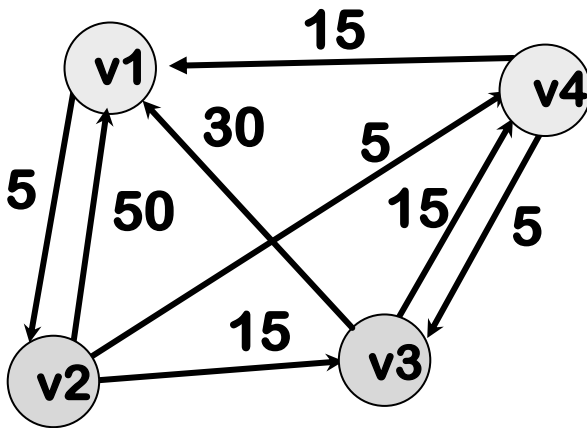


CALCULATING DS



$$D^{(k+1)}[i][j] = \min\{D^{(k)}[i][k+1] + D^{(k)}[k+1][j], D^{(k)}[i][j]\}$$

SHORTEST PATH: EXAMPLE



G



	v1	v2	v3	v4
v1	0	5	∞	∞
v2	50	0	15	5
v3	30	∞	0	15
v4	15	∞	5	0

W

SHORTEST PATH: EXAMPLE

	v1	v2	v3	v4
v1	0	5	∞	∞
v2	50	0	15	5
v3	30	∞	0	15
v4	15	∞	5	0

$$D^{(0)} = W$$



	v1	v2	v3	v4
v1	0	5	∞	∞
v2	50	0	15	5
v3	30	35	0	15
v4	15	20	5	0

$$D^{(1)}$$



SHORTEST PATH: EXAMPLE

	v1	v2	v3	v4
v1	0	5	20	10
v2	50	0	15	5
v3	30	35	0	15
v4	15	20	5	0

$D^{(2)}$



	v1	v2	v3	v4
v1	0	5	20	10
v2	45	0	15	5
v3	30	35	0	15
v4	15	20	5	0

$D^{(3)}$



SHORTEST PATH: EXAMPLE

	v1	v2	v3	v4
v1	0	5	15	10
v2	20	0	10	5
v3	30	35	0	15
v4	15	20	5	0

$$D^{(4)} = D$$

01 knapsack
inversion

FLOYD-WARSHALL ALGORITHM

```
FloydWarshall(W[][],n)
  D=W //initializing  $D^0$ 
  for k=1 to n do //calculating  $D^k$ 
    for i=1 to n do
      for j=1 to n do
         $D[i][j]=\min\{D[i][j], D[i][k]+D[k][j]\}$ 
  return D
```

$$O(n^3)$$

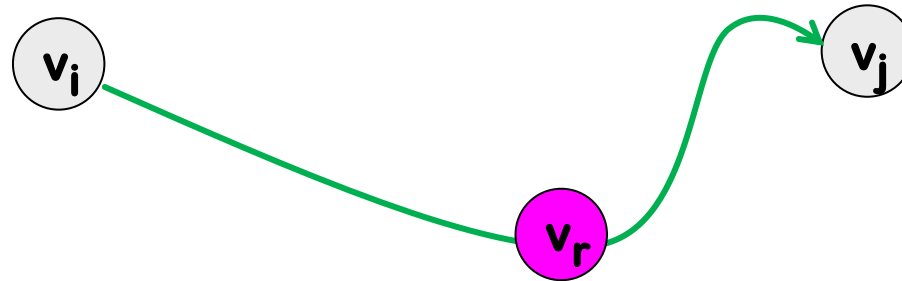
FLOYD-WARSHALL ALGORITHM

```
FloydWarshall(W[][],n)
  D=W //initializing  $D^0$ 
  for k=1 to n do //calculating  $D^k$ 
    for i=1 to n do
      for j=1 to n do
        if  $D[i][k]+D[k][j]<D[i][j]$  then
           $D[i][j]=D[i][k]+D[k][j]$ 
  return D
```

$$O(n^3)$$

SHORTEST PATHS! PATH MATRIX

- Rows and columns of P : v_1 to v_n
- $P[i][j]=r$: shortest path from v_i to v_j passes through v_r



- $P[i][j]=0$: edge $v_i v_j$ is the shortest path from v_i to v_j



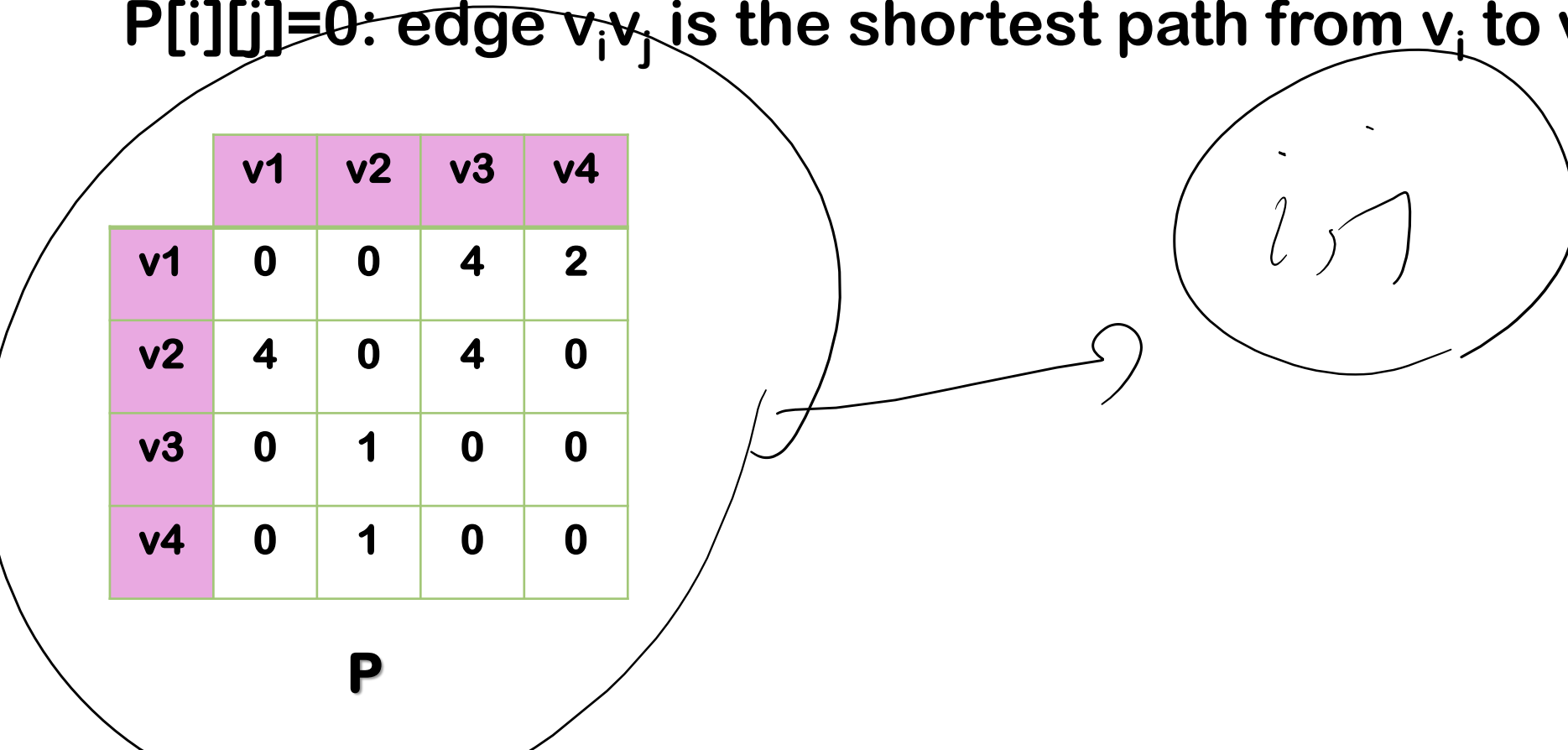
SHORTEST PATHS!

1. How to calculate path matrix, P ?
2. How to use P to find the shortest paths?

SHORTEST PATHS!

$P[i][j]=r$: shortest path from v_i to v_j passes through v_r

$P[i][j]=0$: edge $v_i v_j$ is the shortest path from v_i to v_j



	v1	v2	v3	v4
v1	0	0	4	2
v2	4	0	4	0
v3	0	1	0	0
v4	0	1	0	0

P

SHORTEST PATHS!

$P[i][j]=r$: shortest path from v_i to v_j passes through v_r

$P[i][j]=0$: edge $v_i v_j$ is the shortest path from v_i to v_j

	v1	v2	v3	v4
v1	0	0	4	2
v2	4	0	4	0
v3	0	1	0	0
v4	0	1	0	0

P

$SP(v1, v3)=??$



SHORTEST PATHS!

$P[i][j]=r$: shortest path from v_i to v_j passes through v_r

$P[i][j]=0$: edge $v_i v_j$ is the shortest path from v_i to v_j

	v1	v2	v3	v4
v1	0	0	4	2
v2	4	0	4	0
v3	0	1	0	0
v4	0	1	0	0

P

$SP(v1, v3) = ??$

$P[v1][v3] = ?$



SHORTEST PATHS!

$P[i][j]=r$: shortest path from v_i to v_j passes through v_r

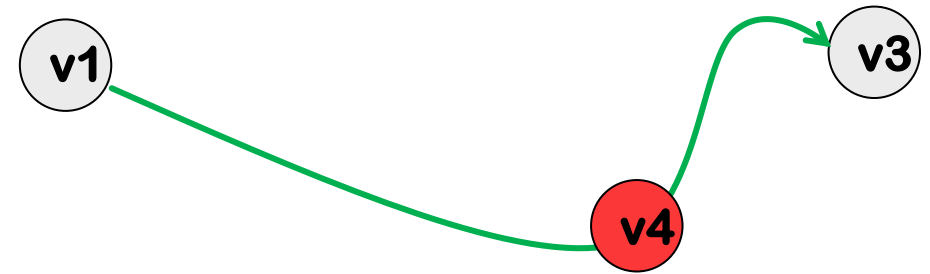
$P[i][j]=0$: edge $v_i v_j$ is the shortest path from v_i to v_j

	v1	v2	v3	v4
v1	0	0	4	2
v2	4	0	4	0
v3	0	1	0	0
v4	0	1	0	0

P

$SP(v1, v3)=??$

$P[v1][v3]=4$



SHORTEST PATHS!

$P[i][j]=r$: shortest path from v_i to v_j passes through v_r

$P[i][j]=0$: edge $v_i v_j$ is the shortest path from v_i to v_j

	v1	v2	v3	v4
v1	0	0	4	2
v2	4	0	4	0
v3	0	1	0	0
v4	0	1	0	0

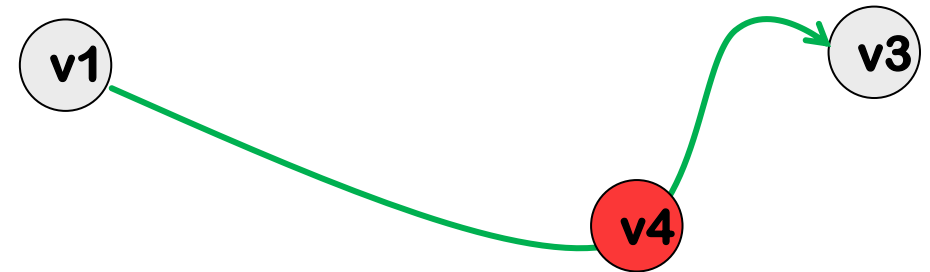
P

$SP(v1, v3) = ??$

$P[v1][v3] = 4$

1. $SP(v1, v4) = ?$

2. $SP(v4, v3) = ?$



SHORTEST PATHS!

$P[i][j]=r$: shortest path from v_i to v_j passes through v_r

$P[i][j]=0$: edge $v_i v_j$ is the shortest path from v_i to v_j

	v1	v2	v3	v4
v1	0	0	4	2
v2	4	0	4	0
v3	0	1	0	0
v4	0	1	0	0

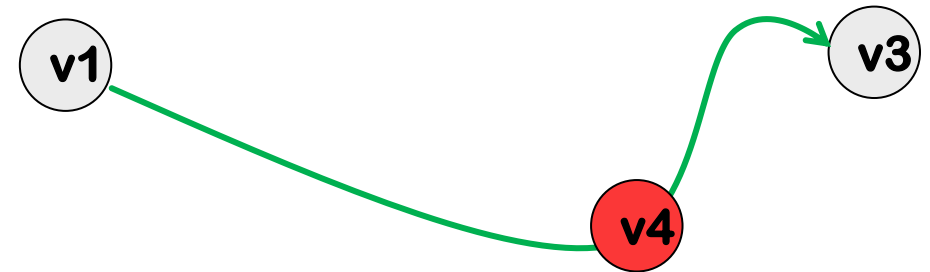
P

$SP(v1, v3) = ??$

$P[v1][v3] = 4$

1. $P[v1][v4] = ?$

2. $P[v4][v3] = ?$



SHORTEST PATHS!

$P[i][j]=r$: shortest path from v_i to v_j passes through v_r

$P[i][j]=0$: edge $v_i v_j$ is the shortest path from v_i to v_j

	v1	v2	v3	v4
v1	0	0	4	2
v2	4	0	4	0
v3	0	1	0	0
v4	0	1	0	0

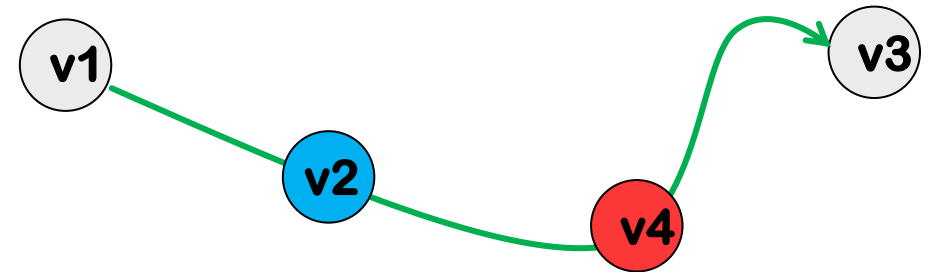
P

$SP(v1, v3) = ??$

$P[v1][v3] = 4$

1. $P[v1][v4] = 2$

2. $P[v4][v3] = 0$



SHORTEST PATHS!

$P[i][j]=r$: shortest path from v_i to v_j passes through v_r

$P[i][j]=0$: edge $v_i v_j$ is the shortest path from v_i to v_j

	v1	v2	v3	v4
v1	0	0	4	2
v2	4	0	4	0
v3	0	1	0	0
v4	0	1	0	0

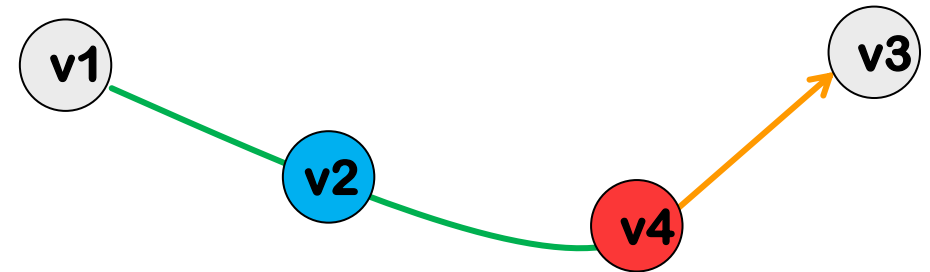
P

$SP(v1, v3) = ??$

$P[v1][v3] = 4$

1. $P[v1][v4] = 2$

2. $P[v4][v3] = 0$



SHORTEST PATHS!

$P[i][j]=r$: shortest path from v_i to v_j passes through v_r

$P[i][j]=0$: edge $v_i v_j$ is the shortest path from v_i to v_j

	v1	v2	v3	v4
v1	0	0	4	2
v2	4	0	4	0
v3	0	1	0	0
v4	0	1	0	0

P

$SP(v1, v3)=??$

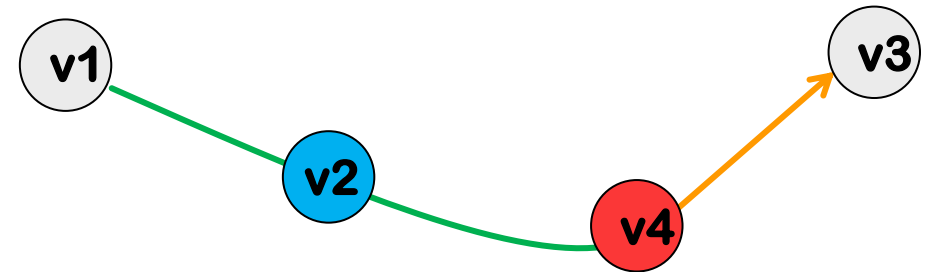
$P[v1][v3]=4$

1. $P[v1][v4]=2$

1.1. $P[v1][v2]=?$

1.2. $P[v2][v4]=?$

2. $P[v4][v3]=0$



SHORTEST PATHS!

$P[i][j]=r$: shortest path from v_i to v_j passes through v_r

$P[i][j]=0$: edge $v_i v_j$ is the shortest path from v_i to v_j

	v1	v2	v3	v4
v1	0	0	4	2
v2	4	0	4	0
v3	0	1	0	0
v4	0	1	0	0

P

$SP(v1, v3)=??$

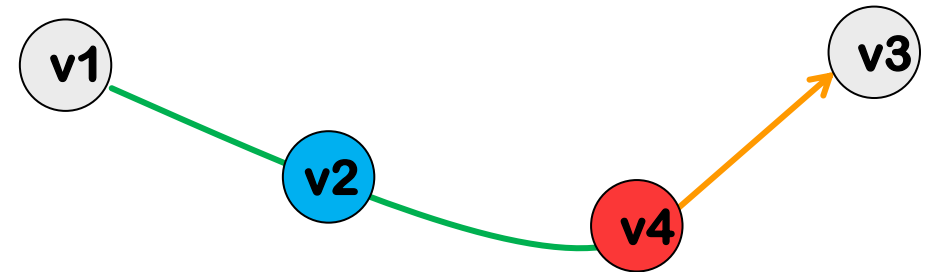
$P[v1][v3]=4$

1. $P[v1][v4]=2$

1.1. $P[v1][v2]=0$

1.2. $P[v2][v4]=0$

2. $P[v4][v3]=0$



SHORTEST PATHS!

$P[i][j]=r$: shortest path from v_i to v_j passes through v_r

$P[i][j]=0$: edge $v_i v_j$ is the shortest path from v_i to v_j

	v1	v2	v3	v4
v1	0	0	4	2
v2	4	0	4	0
v3	0	1	0	0
v4	0	1	0	0

P

$SP(v1, v3)=??$

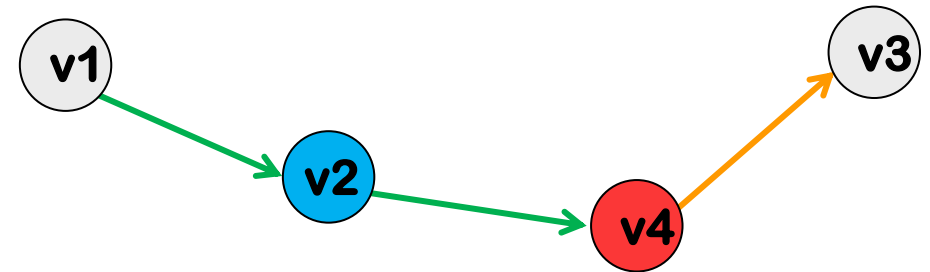
$P[v1][v3]=4$

1. $P[v1][v4]=2$

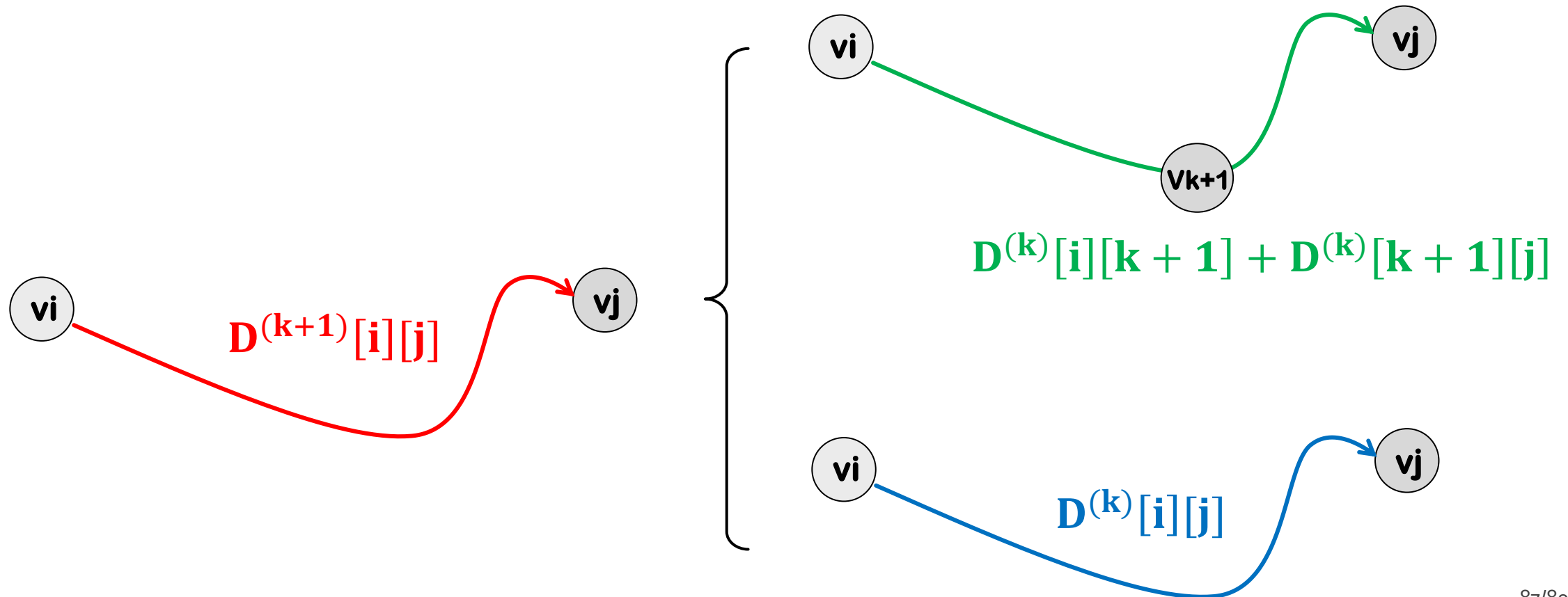
1.1. $P[v1][v2]=0$

1.2. $P[v2][v4]=0$

2. $P[v4][v3]=0$



CALCULATING P



$$D^{(k)}[i][k+1] + D^{(k)}[k+1][j] < D^{(k)}[i][j]: P[i][j] = k+1$$

FLOYD'S ALGORITHM

```
FloydWarshall(W[][],n)
  D=W
  P=0 //initializing matrix P
  for k=1 to n do
    for i=1 to n do
      for j=1 to n do
        if D[i][k]+D[k][j]<D[i][j] then
          D[i][j]=D[i][k]+D[k][j]
          P[i][j]=k
  return D
```

ANY QUESTION??

HAVE A NICE DAY!